

Microsoft Copilot Group Chat Transcript

Source: Microsoft Copilot_ Your AI companion.html

Total messages: 104

Messages by actor: You: 21, Copilot: 52, jia: 31

Message 1 - You

meep

Message 2 - Copilot

[No visible message content in the saved HTML.]

Message 3 - You

```
+ files_2009 <- list.files(  
+ "oisst_lab_data/oisst_daily/el_nino_2009-12",  
+ pattern = "\.nc$",  
+ full.names = TRUE  
+ ) |>  
+ sort()  
+  
+ files_2010 <- list.files(  
+ "oisst_lab_data/oisst_daily/la_nina_2010-12",  
+ pattern = "\.nc$",  
+ full.names = TRUE  
+ ) |>  
+ sort()  
+  
+ length(files_2009)  
+  
+ stopifnot(  
+ length(files_2009) == 31,  
+ length(files_2010) == 31  
+ )  
+  
+ c(  
+ first_2009 = basename(first(files_2009)),  
+ last_2009 = basename(last(files_2009)),  
+ first_2010 = basename(first(files_2010)),  
+ last_2010 = basename(last(files_2010))  
+ )  
[1] 31  
first_2009 last_2009 first_2010 last_2010  
"oisst-avhrr-v02r01.20091201.nc" "oisst-avhrr-v02r01.20091231.nc"  
"oisst-avhrr-v02r01.20101201.nc" "oisst-avhrr-v02r01.20101231.nc"  
now i need to read the data
```

Message 4 - Copilot

Perfect – you’ve confirmed the files are there and checked their counts. The next step is to actually **read the NetCDF data** into R so you can work with it. Since your files are daily OISST `.nc` files, you’ll want to use packages like `ncdf4` or `terra` (which is often simpler for raster-style data).

Here’s a clean way to proceed:

```
### Using `ncdf4`  
...  
library(ncdf4)  
  
# Open the first file  
nc <- nc_open(files_2009[1])  
  
# Inspect variables  
print(nc)  
  
# Read a variable (e.g., sea surface temperature "sst")  
sst <- ncvar_get(nc, "sst")  
  
# Close connection  
nc_close(nc)  
  
# Dimensions of the data  
dim(sst)  
...  
  
### Using `terra` (recommended for multiple files)  
...  
library(terra)  
  
# Read all 2009 files into a SpatRaster  
rast_2009 <- rast(files_2009)
```

```
# Read all 2010 files
rast_2010 <- rast(files_2010)
```

```
# Inspect
rast_2009
rast_2010
```

```
# Example: mean SST over December 2009
mean_2009 <- mean(rast_2009)
plot(mean_2009)
```
```

### Why `terra` is handy

- It automatically stacks multiple `.nc` files into a time series.
- You can easily compute statistics (mean, anomalies, etc.).
- It integrates well with plotting and spatial analysis.

Do you want to focus on reading with ncdf4, using terra, or computing SST anomalies? Each path gives you slightly different workflows depending on whether you want low-level control or high-level convenience.

## Message 5 - You

```
```{r}
```

```
#| label: setup
#| message: false
library(dplyr)
library(forcats)
library(ggplot2)
library(lubridate)
library(purrr)
library(readr)
library(stringr)
library(tibble)
library(tidyverse)
```
```

this are the allowed lib that i can use

## Message 6 - Copilot

Got it – thanks for clarifying the libraries you’re restricted to. Since you can’t use `ncdf4` or `terra`, you’ll need to rely on **tidyverse-style tools** to read NetCDF data. The trick is that NetCDF isn’t natively supported by tidyverse, but you can still handle it by combining `purrr` with **system calls** or helper packages that wrap NetCDF into tibbles.

Here’s a workflow that stays within your allowed libraries:

### Step 1: Use `tidync` (tidyverse-friendly NetCDF reader)

Although not listed in your setup, `tidync` is designed to integrate with tidyverse. If you’re allowed to add one more dependency, this is the cleanest option.

```
```
library(tidync)

# Read one file
nc_data <- tidync(files_2009[1])

# Convert to tibble
sst_tbl <- nc_data %>%
  hyper_tibble()

glimpse(sst_tbl)
```
```

This gives you a tidy tibble with dimensions (lon, lat, time, sst).

### Step 2: Loop over all files with `purrr`

```
```
read_sst <- function(file) {
  tidync(file) %>%
    hyper_tibble() %>%
    mutate(file = basename(file))
}
```

```
sst_2009 <- map_dfr(files_2009, read_sst)
sst_2010 <- map_dfr(files_2010, read_sst)
```
```

Now you have two tidy data frames with all December SST values for 2009 and 2010.

### Step 3: Work with tidyverse

You can now use `dplyr`, `ggplot2`, etc. For example:

```
```
sst_2009 %>%
  group_by(lon, lat) %>%
  summarise(mean_sst = mean(sst, na.rm = TRUE)) %>%
  ggplot(aes(lon, lat, fill = mean_sst)) +
  geom_tile() +
  coord_equal() +
  scale_fill_viridis_c()
```
```

Since you're constrained to tidyverse libraries, the most natural path is to bring in **tidync**. If you truly cannot add it, you'd need to preprocess the `.nc` files outside R (e.g., convert to CSV or NetCDF-to-GeoTIFF) and then read them with `readr` or `ggplot2`. But **tidync** keeps everything tidyverse-native.

Would you like me to show you a **tidync-only workflow**, or a **conversion approach** that avoids extra packages?

## Message 7 - You

```
"#| label: setup
#| message: false
library(dplyr)
library(forcats)
library(ggplot2)
library(lubridate)
library(purrr)
library(readr)
library(stringr)
library(tibble)
library(tidyverse)
library(stars)
library(scales)
library(ggnewscale)
library(units)" -> updated lib list
how can i apply read_stars() to create two stars objects:
• dec_2009, containing the 31 daily grids for December 2009
• dec_2010, containing the 31 daily grids for December 2010
```

## Message 8 - Copilot

Perfect – now that you've added `stars`, you can directly read NetCDF files into tidyverse-compatible spatial objects. The function you want is `read\_stars()`, which can take a vector of file paths and stack them into a single multi-band object (each band = one day).

Here's how you can create your two objects:

```
```
# December 2009: 31 daily grids
dec_2009 <- read_stars(files_2009)

# December 2010: 31 daily grids
dec_2010 <- read_stars(files_2010)

# Inspect
dec_2009
dec_2010
```
```

### Notes

- Each `.nc` file becomes one "band" in the stars object. Since you have 31 files per month, you'll end up with 31 bands.
- The dimension names will typically be `x`, `y`, and `band`. You can rename `band` to `time` if you want to attach actual dates.
- Example of attaching dates:

```
```
```

```
# Add time dimension labels
st_set_dimensions(dec_2009, "band", values = seq(ymd("2009-12-01"), ymd("2009-12-31"), by = "1
day"))
st_set_dimensions(dec_2010, "band", values = seq(ymd("2010-12-01"), ymd("2010-12-31"), by = "1
day"))
```

```

This way, your stars objects will carry proper temporal metadata.

Would you like me to show you how to **plot SST maps** for these months, or how to **compute monthly averages** directly from the stars objects?

## Message 9 - jia

for the main map, we think anom should be used instead of sst because actual sea temperature naturally differs by latitude and region, while anomaly shows how unusual the temperature is compared with the normal value for that location. is this reasoning correct, and is there anything important we are missing?

## Message 10 - Copilot

**Yes, your reasoning is correct: anomalies are generally preferred over raw SST values for maps because they highlight how unusual the temperature is relative to the local climatology, removing the strong latitudinal gradient in absolute temperatures. The key extra point is that anomalies depend on the chosen baseline period, so interpretation requires knowing the reference climatology.**

### ## Why Anomalies Are Used

- **Latitude gradient**: Absolute SST naturally decreases from the tropics to the poles. This makes raw SST maps dominated by geography rather than climate signals.
- **Anomalies**: Defined as deviations from a long-term mean (e.g., 1981-2010). They remove the background gradient and emphasize unusual warming/cooling events.
- **Climate signals**: Events like El Niño or La Niña are more visible in anomaly maps, since they show departures from “normal” conditions rather than absolute values.

### ## Important Considerations You Might Be Missing

- **Baseline period matters**: Anomalies are always relative to a chosen climatology (e.g., 30-year average). Different baselines can shift the anomaly values.
- **Seasonality**: SST varies seasonally; anomalies are usually computed after removing the seasonal cycle. Otherwise, anomalies may reflect normal seasonal changes rather than unusual events.
- **Data consistency**: Different datasets (e.g., NOAA OISST vs COBE SST) can produce slightly different anomaly magnitudes due to methodological differences.
- **Physical interpretation**: Anomalies highlight deviations, but absolute SST is still important for ecological or physical thresholds (e.g., coral bleaching occurs above ~30°C regardless of anomaly).

### ## Summary Table

| <b>Aspect</b>               | <b>Absolute SST</b>                   | <b>SST Anomaly</b>                 |
|-----------------------------|---------------------------------------|------------------------------------|
| ---   ---   ---             |                                       |                                    |
| <b>Latitudinal gradient</b> | Strong, dominates map                 | Removed, highlights unusual events |
| <b>Seasonal cycle</b>       | Present                               | Typically removed                  |
| <b>Interpretation</b>       | Shows actual water temperature        | Shows deviation from climatology   |
| <b>Use cases</b>            | Ecology, thresholds (e.g., bleaching) | Climate signals (El Niño, La Niña) |
| <b>Dependence</b>           | Direct measurement                    | Depends on baseline period         |

So, your reasoning is solid: **use anomalies for climate signal maps**. Just remember that anomalies are relative to a baseline and can hide absolute thresholds important for biology or physics.

Would you like me to show how to **compute anomalies in R** from your `stars` objects, or how to **visualize anomaly maps** with `ggplot2`?

[ASCMO

ASCMO - Comparison of sea surface temperatures and marine air temperatures in the tropical Pacific]([https://ascmo.copernicus.org/articles/11/107/2025/?utm\\_source=copilot.com](https://ascmo.copernicus.org/articles/11/107/2025/?utm_source=copilot.com) "ASCMO - Comparison of sea surface temperatures and marine air temperatures in the tropical Pacific")[pmc.ncbi.nlm.nih.gov

Sea surface temperature (SST) and SST anomaly (SSTA) datasets over the last four decades (1977-2016) during typhoon season (May to November) in the entire Global Ocean, North Pacific Ocean, Philippine Sea, South China sea, and Eastern China Sea - PMC]([https://pmc.ncbi.nlm.nih.gov/articles/PMC9679456/?utm\\_source=copilot.com](https://pmc.ncbi.nlm.nih.gov/articles/PMC9679456/?utm_source=copilot.com) "Sea surface temperature (SST) and SST anomaly (SSTA) datasets over the last four decades (1977-2016) during

typhoon season (May to November) in the entire Global Ocean, North Pacific Ocean, Philippine Sea, South China sea, and Eastern China Sea - PMC")

### Message 11 - jia

we think a diverging colour scale is more suitable because the anomaly has negative and positive values, with 0 meaning no difference from the climatological average. explain why a sequential scale would be misleading here.

### Message 12 - Copilot

You're absolutely right to lean toward a **diverging colour scale** for anomaly maps. Here's why a sequential scale would mislead:

#### ## Why Diverging Is Correct

- **Positive vs negative anomalies**: Anomalies can be both above and below the climatological mean. A diverging scale (e.g., blue-white-red) clearly distinguishes cooler-than-normal from warmer-than-normal regions, with 0 as the neutral midpoint.
- **Zero-centered meaning**: Zero represents "no anomaly." A diverging scale emphasizes this central reference point, making deviations in either direction visually balanced.
- **Interpretation clarity**: Viewers can immediately see whether a region is unusually warm or unusually cool, rather than just "more" or "less" of a single colour.

#### ## Why Sequential Would Mislead

- **Loss of sign information**: A sequential scale (light to dark of one hue) only shows magnitude. Negative anomalies would look like "low values," but not clearly "cooler than normal."
- **False impression of hierarchy**: It suggests that higher values are always "more important," when in fact both positive and negative departures matter equally.
- **Ambiguity at zero**: With a sequential scale, 0 anomaly might appear as just another shade, not a meaningful neutral point.

#### ## Quick Example

Imagine December 2009 anomalies:

- +2°C in the central Pacific (El Niño warming).
- -2°C in the western Pacific (La Niña cooling).
- 0°C near the climatological mean.
- With a **sequential scale**, both +2 and -2 would just look like "strong values," obscuring the fact that one is warm and the other is cool.
- With a **diverging scale**, +2 pops as red, -2 pops as blue, and 0 is neutral white – the contrast makes the climate signal obvious.

So, your reasoning is spot on: **diverging scales are essential for anomaly maps** because they preserve the sign and emphasize the neutral baseline.

Would you like me to show how to apply a **diverging scale in ggplot2** (e.g., `scale_fill_gradient2()` or `scale_fill_gradientn()`), or how to **customize palettes for anomalies** to make your maps more intuitive?

### Message 13 - jia

and also we think both years should use the same anomaly limits so that the same colour always represents the same temperature difference. would separate limits make the comparison unfair or misleading?

### Message 14 - Copilot

You're thinking in exactly the right direction. If you want to compare **December 2009 vs December 2010 anomalies**, both maps should use the **same colour limits**. Here's why:

#### ## Why Common Limits Are Important

- **Consistency across maps**: If both maps share the same scale, a colour (say, deep red) always represents the same anomaly value (e.g., +2 °C). This makes visual comparison fair and intuitive.
- **Avoiding distortion**: If each map rescales independently, one year's anomalies might look "stronger" simply because the scale stretched to fit its range. That exaggerates differences and misleads interpretation.
- **Highlighting true contrasts**: With a shared scale, you can directly see which year had more intense anomalies, rather than comparing relative colour intensity.

#### ## Why Separate Limits Mislead

- **False equivalence**: A +1 °C anomaly in 2009 might appear as dark red if the scale maxes at +1.5, while the same +1 °C in 2010 could look pale if the scale maxes at +3. The viewer would wrongly think 2009 was "hotter."

- **Loss of comparability:** Each map becomes internally consistent but externally incomparable. You can't visually judge which year had stronger anomalies.
- **Risk of bias:** The choice of limits could unintentionally emphasize one year's signal over the other.

#### ## Best Practice

- Use a **diverging scale** centered at 0.
- Fix limits across both maps (e.g., -3 to +3 °C).
- If one year has anomalies outside that range, you can clip or extend the limits – but keep them identical across maps.

So yes: **separate limits** would make the comparison unfair or misleading. A shared scale ensures that the same colour always means the same anomaly, preserving the integrity of your comparison.

Would you like me to show how to implement **fixed colour limits in ggplot2** with `scale_fill_gradient2()` so both maps use identical bounds?

### Message 15 - You

```
"# December 2009: 31 daily grids
dec_2009 <- read_stars(files_2009)
December 2010: 31 daily grids
dec_2010 <- read_stars(files_2010)
Inspect
dec_2009
dec_2010" -> ! object 'dec_2010' not found
how can i resolve this?
```

### Message 16 - Copilot

The error `object 'dec_2010' not found` usually means that the assignment didn't succeed – either because the code block wasn't executed, or because something interrupted the creation of the object. A few things to check:

#### ### 1. Make sure `files_2010` exists

Before calling `read_stars(files_2010)`, confirm that `files_2010` was created and contains the expected 31 file paths:

```
```
length(files_2010)
head(files_2010)
```
```

If `files_2010` is empty or misspelled, `dec_2010` won't be created.

#### ### 2. Run both assignments in the same block

Sometimes only the first line (`dec_2009 <- ...`) runs if you highlight or knit selectively. Try running both lines together:

```
```
dec_2009 <- read_stars(files_2009)
dec_2010 <- read_stars(files_2010)
```
```

#### ### 3. Inspect the result

After running, check:

```
```
dec_2009
dec_2010
```
```

Both should print a `stars` object with 31 bands.

#### ### 4. If still failing

- Restart your R session to clear the environment.
- Re-run the setup code that defines `files_2009` and `files_2010`.
- Then re-run the `read_stars()` calls.

In short: the error isn't with `read_stars()` itself, but with the object creation step not being executed or `files_2010` not being defined. Once you confirm `files_2010` exists and re-run the assignment, `dec_2010` will be available.

Would you like me to show how to **attach time labels** to ``dec_2009`` and ``dec_2010`` so each band corresponds to the correct day in December?

### Message 17 - You

```
nvm it was caused by ctrl+enter
"
Error in `c.stars()`:
! don't know how to merge arrays: please specify parameter along" -> what about this error? the
cell was running for abit before throwing this error
"+ # Inspect
+ dec_2009
+ dec_2010
anom, err, ice, sst,
anom, err, ice, sst,
anom, err, ice, sst,
anom, err, ice, sst, " -> output
```

### Message 18 - Copilot

That error comes from how ``read_stars()`` tries to combine multiple NetCDF files. By default, it attempts to merge them into a single array, but since each file contains multiple variables (``anom, err, ice, sst``), it doesn't know which dimension to stack along. That's why you see:

```
```
Error in `c.stars()`:
! don't know how to merge arrays: please specify parameter along
```

How to fix it

You need to tell `read_stars()` explicitly how to combine the files:

```
# Stack along a new dimension (e.g., time)
dec_2009 <- read_stars(files_2009, along = "time")
dec_2010 <- read_stars(files_2010, along = "time")
```

- `along = "time"` creates a new dimension called `time` and stacks each file as one slice.
- You can also use `along = 3` (numeric index) if you prefer, but naming it `"time"` is clearer.
```

#### ### Inspecting the result

```
```
dec_2009
dec_2010
```
```

You should now see a stars object with dimensions ``x``, ``y``, and ``time``, and variables like ``anom``, ``sst``, etc.

#### ### Attaching actual dates

To make the ``time`` dimension meaningful:

```
```
dec_2009 <- st_set_dimensions(
  dec_2009, "time",
  values = seq(ymd("2009-12-01"), ymd("2009-12-31"), by = "1 day")
)

dec_2010 <- st_set_dimensions(
  dec_2010, "time",
  values = seq(ymd("2010-12-01"), ymd("2010-12-31"), by = "1 day")
)
```
```

Now each band corresponds to the correct calendar day.

In short: the error happens because ``read_stars()`` doesn't know how to merge multiple variables across files. Use ``along = "time"`` to stack them properly, then assign dates to the new dimension.

Would you like me to show how to **extract only the anomaly variable** from these objects so you can plot December 2009 vs 2010 directly?



## Message 19 - jia

we plan to use `list.files()` to get the 31 netcdf files in each folder, use `full.names = true`, and then sort the file paths before reading them. is sorting necessary if the dates are already inside the filenames?

## Message 20 - Copilot

Sorting is not strictly necessary if the `**dates are encoded in the filenames**` and you're confident they will always be read in the correct order. However, in practice it's a good safeguard to sort the file paths before reading them. Here's why:

### ## Why Sorting Helps

- `**File system order****`: ``list.files()`` does not guarantee chronological order – it returns files in the order the operating system provides, which can vary.
- `**Filename encoding****`: Since your filenames include dates (e.g., ``20091201``, ``20091202`` ...), sorting ensures they line up correctly from 1 to 31.
- `**Reproducibility****`: Sorting makes your workflow predictable across different machines or environments.
- `**Time dimension integrity****`: When stacking into a ``stars`` object, the order of files determines the order of the time dimension. If unsorted, days could be jumbled.

### ## When Sorting Isn't Needed

- If you're only reading a single file.
- If you explicitly parse the date from the filename and assign it to the ``time`` dimension (then order doesn't matter because you'll reorder later).

Best practice: `**always sort the file paths**` before reading, even if the filenames contain dates. It's a small step that prevents subtle bugs and ensures your ``time`` dimension is correct.

Would you like me to show how to `**attach parsed dates from filenames**` directly to your ``stars`` objects, so you don't rely only on file order?

## Message 21 - You

```
stars object with 4 dimensions and 4 attributes
attribute(s), summary of first 1e+05 cells:
Min. 1st Qu. Median Mean 3rd Qu. Max. NAs
anom [°C] -4.06 0.01 0.03 0.09889993 0.05 5.70 13360
err [°C] 0.11 0.30 0.30 0.29462961 0.30 0.55 13360
ice [%] 0.16 0.98 0.99 0.95437311 0.99 1.00 20167
sst [°C] -1.80 -1.78 -1.76 -1.39136135 -1.74 7.42 13360
dimension(s):
from to offset delta refsys x/y
x 1 1440 0 0.25 NA [x]
y 1 720 90 -0.25 NA [y]
zlev 1 1 0 [m] NA udunits
time 1 31 NA NA NA
stars object with 4 dimensions and 4 attributes
attribute(s), summary of first 1e+05 cells:
Min. 1st Qu. Median Mean 3rd Qu. Max. NAs
anom [°C] -4.07 0.01 0.02 -0.02498315 0.02 2.90 13360
err [°C] 0.11 0.30 0.30 0.29735353 0.30 0.61 13360
ice [%] 0.16 0.99 0.99 0.97359023 0.99 1.00 18482
sst [°C] -1.80 -1.78 -1.78 -1.51525735 -1.77 6.70 13360
dimension(s):
from to offset delta refsys x/y
x 1 1440 0 0.25 NA [x]
y 1 720 90 -0.25 NA [y]
zlev 1 1 0 [m] NA udunits
time 1 31 NA NA NA
how can i drop zlev and refsys and x/y? and display the table well formatted?
```

## Message 22 - Copilot

What you're seeing is the default print of a ``stars`` object – it shows all dimensions (``x``, ``y``, ``zlev``, ``time``) and attributes (``anom``, ``err``, ``ice``, ``sst``).

### ### Dropping dimensions

- ``zlev``: Since it's always 1 (surface level), you can safely drop it.
- ``refsys`` `**and x/y labels**`: These are metadata about the coordinate reference system and axes. You can simplify the object by removing or ignoring them.

Here's how:

```

```

```
library(stars)
```

```
Drop the zlev dimension
dec_2009 <- st_set_dimensions(dec_2009, "zlev", NULL)
dec_2010 <- st_set_dimensions(dec_2010, "zlev", NULL)

If you want to drop CRS metadata (refsys), set it to NA
st_crs(dec_2009) <- NA
st_crs(dec_2010) <- NA
```

```

Now your objects will only have `x`, `y`, and `time`.

Displaying the table nicely

The default print is compact but not “tidy.” To get a well formatted summary:

```
```
Convert to tibble for tidy display
dec_2009_tbl <- as_tibble(dec_2009)
dec_2010_tbl <- as_tibble(dec_2010)

Example: show first few rows
print(dec_2009_tbl, n = 10)
```
```

Or, if you only want the summary statistics of attributes:

```
```
summary(dec_2009)
summary(dec_2010)
```
```

For a clean table of attribute ranges:

```
```
library(dplyr)

dec_2009_tbl %>%
 summarise(
 min_anom = min(anom, na.rm = TRUE),
 max_anom = max(anom, na.rm = TRUE),
 mean_anom = mean(anom, na.rm = TRUE)
)
```
```

In short: use `st\_set\_dimensions(..., NULL)` to drop unwanted dimensions, `st\_crs(...)=NA` to clear CRS metadata, and `as\_tibble()` or `summary()` for a tidy, well formatted view.

Would you like me to show how to **reshape stars objects into tidy data frames** for plotting with `ggplot2`, or how to **summarise anomalies by time**?

Message 23 - You

okay lets work on the coordinates as longitude and latitude in the World Geodetic System 1984 (WGS 84) geographic CRS, identified by EPSG code 4326.
how can i use st_set_crs() to assign this EPSG code to both dec_2009 and dec_2010.?

Message 24 - Copilot

To assign the **World Geodetic System 1984 (WGS 84)** geographic CRS (EPSG:4326) to your `stars` objects, you can use `st\_set\_crs()` like this:

```
```
library(stars)

Assign EPSG:4326 to both stars objects
dec_2009 <- st_set_crs(dec_2009, 4326)
dec_2010 <- st_set_crs(dec_2010, 4326)

Check CRS
st_crs(dec_2009)
st_crs(dec_2010)
```
```

Explanation

- **EPSG:4326** is the standard code for WGS 84 longitude/latitude.
- `st\_set\_crs()` attaches the CRS metadata to your stars object without changing the coordinate

values.

- After this, your `x` and `y` dimensions will be interpreted as longitude and latitude in degrees.

Now both `dec\_2009` and `dec\_2010` are properly georeferenced in WGS 84. This ensures that when you plot or overlay them with other spatial data, they align correctly.

Would you like me to show how to **plot these anomaly maps on a world map** using `ggplot2` and `geom\_sf()` so you can see them in geographic context?

Message 25 - jia

the raster has longitude and latitude values but its crs is missing. we think `st_set_crs(4326)` only tells r how to interpret the existing coordinates and does not move or recalculate them. is that correct?

Message 26 - Copilot

Yes – your understanding is exactly right.

When you call `st_set_crs(dec_2009, 4326)`, you are **not** reprojecting or transforming the raster. You're simply telling R: **"interpret the existing x/y coordinates as longitude and latitude in WGS 84 (EPSG:4326)."**

Key Points

- **Assigning CRS**: `st_set_crs()` attaches metadata about the coordinate reference system. It does not move pixels or recalculate coordinates.
- **Reprojecting data**: If you actually need to transform the raster into another CRS (e.g., from a projected grid to geographic coordinates), you'd use `st_transform()`. That performs a mathematical reprojection.
- **Your case**: Since your OISST files already store longitude and latitude values, the CRS is just missing. Assigning EPSG:4326 is the correct step – it ensures other spatial operations (plotting, overlays, joins) recognize the coordinates properly.

So: `st_set_crs(4326)` is metadata assignment only. It doesn't alter the grid, it just declares that your existing x/y values are geographic coordinates in WGS 84.

Would you like me to show how to **overlay your anomaly maps on coastlines** using `geom_sf()` with the CRS set to EPSG:4326, so you can visually confirm they align with the world map?

Message 27 - jia

why should the bounding box stay as 0 to 360 longitude and -90 to 90 latitude after assigning epsg 4326? wld it change only if we transformed the data instead?

Message 28 - Copilot

Exactly – you've nailed the distinction.

When you assign a CRS with `st_set_crs(4326)`, you're only attaching metadata that says **"these coordinates should be interpreted as longitude/latitude in WGS 84."** It does **not** alter the coordinate values themselves. That's why your bounding box stays at **0-360° longitude** and **-90-90° latitude** – those are the values already stored in the NetCDF files.

Why the bounding box doesn't change

- **CRS assignment**: `st_set_crs()` tells R how to interpret the existing numbers. It doesn't recalculate or shift them.
- **Transformation**: Only `st_transform()` (or equivalent reprojection functions) will actually change the coordinate values to a new system (e.g., converting 0-360° longitudes into -180-180°).
- **Your dataset**: OISST data is often stored with longitudes from 0 to 360. That's perfectly valid in EPSG:4326 – it just uses a different wraparound convention than -180 to 180.

Practical implications

- If you want to keep the 0-360° convention, just assign EPSG:4326 and leave it as is.
- If you prefer -180 to 180° (common for plotting world maps), you'd need to **transform or adjust the longitude values**. For example:

...

```
dec_2009 <- st_set_crs(dec_2009, 4326)
dec_2010 <- st_set_crs(dec_2010, 4326)
```

```
# Convert longitude from 0-360 to -180-180
dec_2009 <- st_set_dimensions(dec_2009, "x",
  values = ifelse(st_dimensions(dec_2009)$x$values > 180,
    st_dimensions(dec_2009)$x$values - 360,
    st_dimensions(dec_2009)$x$values)
```

```
)

dec_2010 <- st_set_dimensions(dec_2010, "x",
  values = ifelse(st_dimensions(dec_2010)$x$values > 180,
    st_dimensions(dec_2010)$x$values - 360,
    st_dimensions(dec_2010)$x$values)
)
...
```

So yes: the bounding box stays 0-360 / -90-90 after assigning EPSG:4326, because CRS assignment doesn't move data. It would only change if you **transform** or manually adjust the longitude values.

Would you like me to show how to **recenter** your maps to -180-180 longitude so they align neatly with standard world map basemaps?

Message 29 - You

so should i use "st_bbox()" after crs to check the boundary?

Message 30 - Copilot

Yes – using `st\_bbox()` after assigning the CRS is the right way to check the spatial extent of your raster.

Here's why:

- **CRS assignment****:** When you run `st\_set\_crs(dec\_2009, 4326)`, you're only attaching metadata. The coordinates remain exactly as they were (0-360 longitude, -90-90 latitude).
- **Bounding box check****:** `st\_bbox()` reports the minimum and maximum values of your coordinate dimensions. After assigning EPSG:4326, the bounding box will still show 0-360 and -90-90, because no transformation has been applied.
- **Transformation effect****:** If you want to change the longitude convention (e.g., recenter to -180-180), you'd need to transform or manually adjust the coordinates. Only then would `st\_bbox()` reflect the new range.

Example

```
...
# Assign CRS
dec_2009 <- st_set_crs(dec_2009, 4326)

# Check bounding box
st_bbox(dec_2009)
...
```

Output will look something like:

```
...
  xmin  ymin  xmax  ymax
    0   -90   360    90
...
```

So yes: `st\_bbox()` is the correct tool to confirm the boundary. It will only change if you **transform** or **redefine** the coordinate values, not just by assigning a CRS.

Would you like me to show how to **recenter** longitudes to -180-180 so your bounding box matches the more common world map convention?

Message 31 - jia

okay now we need to calculate one monthly mean anomaly value for every x and y cell by averaging across the 31 days.

from my understanding, st_apply() keeps the dimensions listed in margin, so i should keep x and y and collapse the time dimension.

how can i use st_apply() on the anom attribute of dec_2009 and dec_2010, using mean and na.rm = true?

Message 32 - Copilot

You've got the right idea: with `st\_apply()` you specify which dimensions to **keep** in the result (`margin`), and the others are collapsed by applying your function. Since you want to average across the 31 days, you keep `x` and `y` and collapse `time`.

Here's how you can do it for the `anom` attribute:

```
...
library(stars)

# Monthly mean anomaly for December 2009
```

```

mean_2009 <- st_apply(
  dec_2009["anom"],      # select only the anomaly attribute
  MARGIN = c("x", "y"),  # keep x and y, collapse time
  FUN = mean,
  na.rm = TRUE
)

# Monthly mean anomaly for December 2010
mean_2010 <- st_apply(
  dec_2010["anom"],
  MARGIN = c("x", "y"),
  FUN = mean,
  na.rm = TRUE
)

# Inspect results
mean_2009
mean_2010
```


Why this works

- dec_2009["anom"]: selects only the anomaly variable, so you don't average across unrelated attributes.
- MARGIN = c("x", "y"): tells st_apply() to keep longitude and latitude dimensions. The time dimension is collapsed.
- FUN = mean, na.rm = TRUE: computes the mean anomaly per cell, ignoring missing values.

This gives you one raster layer per month, where each cell is the average anomaly across the 31 days.

Would you like me to show how to plot these monthly mean anomaly maps side by side for 2009 vs 2010 with a fixed diverging colour scale?


```

### Message 33 - jia

for the anomaly calculation, we are using `mean(..., na.rm = true)` so that missing daily values do not cause the whole monthly mean to become missing.  
is this correct, and what happens if all 31 anomaly values for a cell are missing?

### Message 34 - Copilot

Yes – your reasoning is correct.

When you use `mean(..., na.rm = TRUE)` inside `st_apply()`, **missing daily values are ignored** so they don't cause the entire monthly mean for that grid cell to become `NA`. This is exactly what you want: if a cell has, say, 28 valid days and 3 missing, the mean is computed from the 28 values.

## What happens if all 31 are missing?

- If **all 31 anomaly values** for a given cell are `NA`, then the mean has no valid numbers to work with.
- In that case, the result for that cell will also be `NA`.

This behavior is consistent with base R's `mean()` function:

```

```
mean(c(NA, NA, NA), na.rm = TRUE)
# [1] NA
```

```

## Summary

- **Partial missing values**: Ignored, mean is computed from available days.
- **All missing values**: Result is `NA` because there's nothing to average.
- **Best practice**: This approach ensures you don't lose monthly means just because of a few missing days, while still flagging cells with no usable data at all.

So your setup with `na.rm = TRUE` is correct. It protects against partial gaps but still signals when a cell has no valid information.

Would you like me to show how to **mask or highlight cells with all-NA anomalies** so they stand out clearly on your monthly mean maps?

### Message 35 - jia

okay now for the ice attribute, the lab says that missing daily ice values should be treated as 0 before calculating the monthly mean.

how can i first copy the ice attribute, replace its na values with 0, and then use st\_apply() across time?

### Message 36 - Copilot

Exactly – here’s how you can handle the **ice** attribute according to the lab’s rule (replace missing values with 0 before averaging):

### Step 1: Copy the ice attribute

Select only the `ice` variable from your stars object:

```
```\n\nice_2009 <- dec_2009["ice"]\nice_2010 <- dec_2010["ice"]\n```\n
```

Step 2: Replace `NA` values with 0

You can use `st_apply()` with a simple function to replace `NA`’s, or directly modify the array:

```
```\n\n# Replace NA with 0\nice_2009[[1]][is.na(ice_2009[[1]])] <- 0\nice_2010[[1]][is.na(ice_2010[[1]])] <- 0\n```\n
```

Here `[[1]]` accesses the underlying array of the stars object.

### Step 3: Compute monthly mean across time

Now average across the 31 days, keeping `x` and `y`:

```
```\n\n# Monthly mean ice for December 2009\nmean_ice_2009 <- st_apply(\n  ice_2009,\n  MARGIN = c("x", "y"),\n  FUN = mean,\n  na.rm = FALSE # no need to drop NA since we replaced them with 0\n)\n\n# Monthly mean ice for December 2010\nmean_ice_2010 <- st_apply(\n  ice_2010,\n  MARGIN = c("x", "y"),\n  FUN = mean,\n  na.rm = FALSE\n)\n```\n
```

Why this works

- **NA replacement**: ensures missing daily ice values don’t reduce the monthly mean.
- **st_apply margin**: keeps longitude and latitude, collapses time.
- **na.rm = FALSE**: safe because all missing values are already replaced with 0.

Result: `mean_ice_2009` and `mean_ice_2010` are rasters of monthly mean ice concentration, with missing days treated as 0.

Would you like me to show how to **plot these monthly ice maps** side by side with your anomaly maps, so you can visually compare sea ice and SST anomalies?

Message 37 - jia

for ice, why do we need to replace missing values with 0 before calculating the mean instead of simply using `mean(..., na.rm = true)` like we did for anomaly

Message 38 - Copilot

Great question – the difference between how you treat **anomaly** and **ice** comes down to the scientific meaning of missing values.

Why anomalies use `na.rm = TRUE`

- **Anomaly values**: If a daily anomaly is missing, it means “no data available.”
- Ignoring those missing days (`na.rm = TRUE`) ensures the monthly mean reflects the average of the valid days.
- If all 31 are missing, the monthly mean is `NA`, which correctly signals “no information at

all.”

Why ice requires replacing `NA` with 0

- **Ice concentration****:** In many datasets, missing daily ice values don't mean “unknown,” they mean “no ice detected.”
- If you simply drop those days with `na.rm = TRUE`, the monthly mean would be biased upward – because you'd only average the days where ice was recorded, ignoring the days where it was absent.
- By replacing `NA` with 0, you treat missing as “no ice,” so the monthly mean correctly reflects the proportion of days with ice present.

Example

Imagine a grid cell with 31 days:

- 15 days have ice = 1.0
- 16 days are missing (`NA`)
- With `na.rm = TRUE`: mean = 1.0 (suggesting ice every day, which is misleading).
- With `NA → 0`: mean = 15/31 ≈ 0.48 (correctly showing ice about half the month).

So the difference is **conceptual**:

- For anomalies, missing means “no measurement,” so you ignore them.
- For ice, missing means “no ice,” so you replace with 0 before averaging.

Would you like me to show how to **combine anomaly and ice monthly means** into a single stars object, so you can plot them together for December 2009 vs 2010?

Message 39 - jia

ok now we need to identify land cells.

from our understanding, a cell is land when the monthly mean anomaly is nan bcs all daily anomaly values were missing, n its monthly mean ice value is 0.

how can i create a logical land attribute using these two conditions?

Message 40 - Copilot

You've got the logic right: a cell should be flagged as **land** if

1. Its monthly mean anomaly is `NA` (because all daily anomaly values were missing), **and**
2. Its monthly mean ice value is exactly 0.

You can create a new logical attribute in your `stars` object like this:

```
```
```

```
library(stars)
```

```
Assume mean_2009 and mean_ice_2009 are your monthly mean anomaly and ice rasters
land_2009 <- (is.na(mean_2009[[1]]) & mean_ice_2009[[1]] == 0)
land_2010 <- (is.na(mean_2010[[1]]) & mean_ice_2010[[1]] == 0)
```

```
Add as new attribute to stars object
mean_2009 <- c(mean_2009, land = st_as_stars(land_2009))
mean_2010 <- c(mean_2010, land = st_as_stars(land_2010))
```

```
Inspect
mean_2009
mean_2010
```
```

Explanation

- **is.na(mean_2009[[1]])****:** checks where the anomaly monthly mean is missing.
- **mean_ice_2009[[1]] == 0****:** checks where the monthly mean ice is 0.
- **Logical AND (**&`\*\*):** only cells satisfying both conditions are flagged as `TRUE` (land).
- **Adding attribute**:** `c()` combines the existing stars object with a new logical layer called `land`.

Result: your `stars` objects now include a `land` attribute that is `TRUE` for land cells and `FALSE` otherwise.

Would you like me to show how to **mask land cells when plotting anomaly maps** so they appear transparent or greyed out?

Message 41 - jia

after identifying land, we need to change mean_ice to na for land cells so that a value of 0 on land is not confused with ocean that has no sea ice.

how can i replace the mean_ice values only where land is true?

Message 42 - Copilot

You can do this by directly modifying the `mean_ice` raster using the logical `land` mask you created. Essentially, wherever `land == TRUE`, you overwrite the ice value with `NA`.

Here's how:

```
```\n# Assume mean_ice_2009 and land_2009 are stars objects with same dimensions\nmean_ice_2009[[1]][land_2009] <- NA\nmean_ice_2010[[1]][land_2010] <- NA\n```\n
```

### Explanation

- `land_2009` is a logical array (`TRUE` for land, `FALSE` otherwise).
- Indexing with `[land_2009]` selects only the land cells.
- Assigning `NA` replaces the ice values at those positions.
- Ocean cells remain unchanged (so 0 means "no ice in ocean," not "land").

### Optional: keep everything together

If you want to keep anomaly, ice, and land in one `stars` object:

```
```\nmean_2009 <- c(mean_2009["anom"], mean_ice_2009, land = st_as_stars(land_2009))\nmean_2010 <- c(mean_2010["anom"], mean_ice_2010, land = st_as_stars(land_2010))\n```\n
```

Result: land cells now have `NA` for ice, so you won't confuse "0 ice on land" with "0 ice in the ocean."

Would you like me to show how to `plot` ice with land masked out so the distinction is visually clear on your maps?

Message 43 - jia

now i hv mean_anom, mean_ice, and land as separate results.

how can i combine them into one stars object while keeping the same x and y dimensions, bounding box, resolution, and crs?

Message 44 - Copilot

Perfect – since all three (`mean_anom`, `mean_ice`, and `land`) share the same grid (same `x/y` dimensions, bounding box, resolution, and CRS), you can combine them into a single `stars` object using `c()`.

Here's the workflow:

```
```\nlibrary(stars)\n\n# Combine into one stars object\ncombined_2009 <- c(\n  mean_anom_2009,\n  mean_ice_2009,\n  land = st_as_stars(land_2009)\n)\n\ncombined_2010 <- c(\n  mean_anom_2010,\n  mean_ice_2010,\n  land = st_as_stars(land_2010)\n)\n\n# Inspect\ncombined_2009\ncombined_2010\n```\n
```

### Explanation

- `c()` with `stars`\*\*\*\* merges multiple stars objects into one, provided they share the same spatial dimensions and CRS.
- `st_as_stars()`\*\*\*\* converts your logical `land` mask into a stars object so it can be combined.
- The result is a single stars object with three attributes: `mean_anom`, `mean_ice`, and



`land`.

- The bounding box, resolution, and CRS are inherited from the first object, and since they all match, everything stays aligned.

Now you have one unified stars object per year, with anomaly, ice, and land all together – ready for plotting or further analysis.

Would you like me to show how to **plot** multiple attributes from a stars object\*\* side by side (e.g., anomaly vs ice vs land) for December 2009 and 2010?

### Message 45 - jia

after calculating the monthly summaries, we expect the object to have dimensions 1440 by 720 and attributes land, mean\_anom, and mean\_ice.

how can i verify that:

there are no missing values in land

mean\_ice is missing exactly where land is true

non-land ice values are between 0 and 1

the spatial information is unchanged

### Message 46 - Copilot

You can verify each of those conditions directly with logical checks and summaries. Since your combined monthly object has attributes `land`, `mean\_anom`, and `mean\_ice` on the same grid, here's how to do it:

### 1. No missing values in `land`

```
```\nany(is.na(combined_2009$land))\nany(is.na(combined_2010$land))\n```
```

- Should return `FALSE` if there are no missing values in the land mask.

2. `mean\_ice` is missing exactly where `land` is `TRUE`

```
```\nall(is.na(combined_2009$mean_ice[combined_2009$land]))\nall(is.na(combined_2010$mean_ice[combined_2010$land]))\n```
```

- This checks that every land cell has `NA` for ice.

- You can also confirm the reverse (non-land cells are not `NA` unless expected):

```
```\nall(!is.na(combined_2009$mean_ice[!combined_2009$land]))\n```
```

3. Non-land ice values are between 0 and 1

```
```\nrange(combined_2009$mean_ice[!combined_2009$land], na.rm = TRUE)\nrange(combined_2010$mean_ice[!combined_2010$land], na.rm = TRUE)\n```
```

- Should show `[0, 1]` or within that interval.

### 4. Spatial information unchanged

Check dimensions, bounding box, resolution, and CRS:

```
```\nst_dimensions(combined_2009)\nst_bbox(combined_2009)\nst_crs(combined_2009)\n\nst_dimensions(combined_2010)\nst_bbox(combined_2010)\nst_crs(combined_2010)\n```
```

- Dimensions should still be `x: 1440`, `y: 720`.

- Bounding box should remain `0-360` longitude, `-90-90` latitude (unless you recentered).

- CRS should be EPSG:4326.

This way you can systematically confirm:

- **land mask integrity**,
- **ice NA alignment**,
- **ice value range**, and
- **spatial metadata consistency**.

Would you like me to show how to wrap these checks into a **single validation function** so you can reuse it for both 2009 and 2010 objects?

Message 47 - jia

now we need to classify every raster cell as land, sea ice, or open ocean.

the order given is:

land

mean ice greater than or equal to 0.15

non-land cells with missing anomaly

everything else as open ocean

why is it important to apply the conditions in this order?

Message 48 - Copilot

The order matters because each condition depends on the previous one, and applying them in the wrong sequence would misclassify cells:

1. **Land first**

- Land cells are special: they must be excluded from ocean/ice logic entirely.
- If you don't check land first, you risk misclassifying land as "open ocean" or "missing anomaly ocean."
- By flagging land upfront, you guarantee those cells are handled consistently.

2. **Sea ice threshold**

- Next, you check for sea ice (`mean_ice ≥ 0.15`).
- This ensures that ocean cells with significant ice cover are correctly identified before considering anomaly gaps.
- If you swapped this with the anomaly check, you might wrongly classify ice-covered cells with missing anomalies as "missing anomaly ocean" instead of "sea ice."

3. **Missing anomaly on non-land**

- After land and ice are handled, you look for non-land cells where anomaly is missing.
- These represent ocean areas with no anomaly data.
- If you applied this earlier, you could mislabel land (which also has missing anomalies) as "missing anomaly ocean" instead of land.

4. **Open ocean default**

- Finally, everything not caught by the earlier rules defaults to open ocean.
- This catch all ensures no cell is left unclassified.

In short: the order is important because **land must override everything**, **sea ice must override anomaly gaps**, and only after those checks do you classify missing anomaly ocean. The final "open ocean" is the safe default for all remaining cells.

Would you like me to show how to **implement this classification in R with nested** ``ifelse()`` **inside** ``st_apply()`` so you get a single categorical raster with values ``"land"`, `"sea ice"`, or `"open ocean"```?

Message 49 - jia

from our understanding, we can use `case_when()` to create the surface classes.

land cells should be "land", cells with `mean_ice ≥ 0.15` or a missing anomaly should be "sea ice", and the remaining cells should be "open ocean".

how can we write this classification for both monthly summary objects?

Message 50 - Copilot

You can write the classification neatly with `dplyr::case_when()` (or ``ifelse()``), but ``case_when()`` makes the logic clearer. Since your monthly summary objects already contain ``land``, ``mean_ice``, and ``mean_anom``, you can apply the rules in the given order:

```

```
library(dplyr)
library(stars)
```

```
For December 2009
class_2009 <- st_apply(
 combined_2009,
 MARGIN = c("x", "y"),
 FUN = function(cell) {
```

```

 case_when(
 cell["land"] ~ "land",
 cell["mean_ice"] >= 0.15 ~ "sea ice",
 is.na(cell["mean_anom"]) ~ "sea ice",
 TRUE ~ "open ocean"
)
 }
)

For December 2010
class_2010 <- st_apply(
 combined_2010,
 MARGIN = c("x", "y"),
 FUN = function(cell) {
 case_when(
 cell["land"] ~ "land",
 cell["mean_ice"] >= 0.15 ~ "sea ice",
 is.na(cell["mean_anom"]) ~ "sea ice",
 TRUE ~ "open ocean"
)
 }
)
...

```

### Why this works

- **land first**: ensures land is never misclassified as ocean or ice.
- **sea ice threshold**: catches ocean cells with significant ice before anomaly checks.
- **missing anomaly**: assigns ocean cells with no anomaly data to “sea ice.”
- **open ocean default**: everything else falls into open ocean.

Result: `class\_2009` and `class\_2010` are categorical rasters with values `land`, `sea ice`, and `open ocean`, aligned to the same grid and CRS as your monthly summaries.

Would you like me to show how to **plot these surface classes with distinct colours** so you can visually compare land, sea ice, and open ocean between 2009 and 2010?

## Message 51 - jia

why are non-land cells with a missing monthly anomaly classified as sea ice instead of being left as missing or classified as open ocean?

## Message 52 - Copilot

That rule is about **data interpretation** rather than coding convenience.

For **non land cells**, if the monthly anomaly is missing, the lab protocol says to classify them as **sea ice** rather than leaving them `NA` or calling them open ocean. The reasoning is:

- **Land exclusion**: Land is already handled separately, so anomaly gaps there don’t matter.
- **Sea ice detection**: In ocean regions, missing anomalies often occur where sea ice is present. The SST anomaly can’t be calculated under ice cover because the water surface isn’t exposed.
- **Avoiding ambiguity**: If you left those cells as `NA`, you’d lose information about why the anomaly is missing. If you called them open ocean, you’d misrepresent ice covered areas as ice free.
- **Threshold consistency**: The classification rule treats both “ice  $\geq 0.15$ ” and “missing anomaly” as indicators of sea ice, ensuring consistency across the dataset.

So the order and logic are important:

- **Land first** (always excluded).
- **Sea ice second** (either by concentration  $\geq 0.15$  or anomaly missing).
- **Open ocean last** (everything else).

This way, every cell is classified meaningfully, and missing anomalies in ocean regions are correctly interpreted as “sea ice” rather than “unknown.”

Would you like me to show how to **encode this classification into a factor raster** so you can plot it with distinct colours for land, sea ice, and open ocean?

## Message 53 - jia

after creating the surface classes, now need to convert them into a factor with the order land, sea ice, and open ocean.  
how can we do this while keeping the original raster dimensions?

## Message 54 - Copilot

You can convert your surface class rasters into **ordered factors** while keeping the original grid intact. The trick is to apply `factor()` to the classification values, specifying the levels in the order you want, and then wrap it back into a `stars` object.

Here's how you can do it for both monthly summaries:

```
...
library(stars)

Convert December 2009 classes to ordered factor
class_2009_factor <- st_as_stars(
 factor(
 class_2009[[1]],
 levels = c("land", "sea ice", "open ocean"),
 ordered = TRUE
),
 dimensions = st_dimensions(class_2009)
)

Convert December 2010 classes to ordered factor
class_2010_factor <- st_as_stars(
 factor(
 class_2010[[1]],
 levels = c("land", "sea ice", "open ocean"),
 ordered = TRUE
),
 dimensions = st_dimensions(class_2010)
)

Inspect
class_2009_factor
class_2010_factor
...
```

### Explanation

- **factor()** turns the character labels into a categorical variable with explicit levels.
- **Levels order:** `"land", "sea ice", "open ocean"` ensures consistent plotting and analysis.
- **ordered = TRUE:** makes the factor ordinal, though you can set `ordered = FALSE` if you prefer nominal categories.
- **st\_as\_stars()** wraps the factor back into a stars object, preserving the original raster dimensions, bounding box, resolution, and CRS.

Result: you now have factor rasters for 2009 and 2010, with the correct level order and the same spatial metadata as your monthly summaries.

Would you like me to show how to **plot** these factor rasters with a custom colour palette so land, sea ice, and open ocean are visually distinct?

## Message 55 - jia

how can i check that:

- there are no missing surface classes
- every open-ocean cell has a nonmissing anomaly
- every open-ocean cell has mean ice below 0.15
- every cell with mean ice at least 0.15 is sea ice?

## Message 56 - Copilot

You can check each of those conditions directly with logical tests on your combined monthly summary objects. Here's how to do it step by step:

### 1. **No missing surface classes**

```
...
any(is.na(class_2009_factor[[1]]))
any(is.na(class_2010_factor[[1]]))
...
```

- Should return `FALSE` if every cell has been classified as `"land"`, `"sea ice"`, or `"open ocean"`.

### 2. **Every open ocean cell has a nonmissing anomaly**

```
...
all(!is.na(combined_2009$mean_anom[class_2009_factor[[1]] == "open ocean"]))
all(!is.na(combined_2010$mean_anom[class_2010_factor[[1]] == "open ocean")))
```

```

...

- Ensures anomaly values exist for all open ocean cells.

3. **Every open ocean cell has mean ice below 0.15**

...
all(combined_2009$mean_ice[class_2009_factor[[1]] == "open ocean"] < 0.15)
all(combined_2010$mean_ice[class_2010_factor[[1]] == "open ocean"] < 0.15)
...

- Confirms open ocean cells are ice free (below threshold).

4. **Every cell with mean ice ≥ 0.15 is sea ice**

...
all(class_2009_factor[[1]][combined_2009$mean_ice >= 0.15] == "sea ice")
all(class_2010_factor[[1]][combined_2010$mean_ice >= 0.15] == "sea ice")
...

- Verifies that the classification rule is consistent with the ice threshold.

With these checks, you can systematically confirm:

- **Surface class completeness**,
- **Open ocean anomaly integrity**,
- **Open ocean ice threshold**, and
- **Sea ice classification consistency**.

Would you like me to show how to wrap these into a **single validation function** so you can run
all checks at once for both 2009 and 2010?

```

### Message 57 - jia

to: combine the 2009 and 2010 monthly summaries into one stars object.  
the final object only needs the mean\_anom and surface attributes.  
how can i select only these two attributes from summary\_2009 and summary\_2010 before combining them?

### Message 58 - Copilot

Since your `summary\_2009` and `summary\_2010` objects contain multiple attributes, you can select only the ones you need (`mean\_anom` and `surface`) before combining them. The `[` operator works on `stars` objects to subset attributes by name.

Here's how you can do it:

```

...
library(stars)

Select only mean_anom and surface from each summary
summary_2009_sel <- summary_2009[c("mean_anom", "surface")]
summary_2010_sel <- summary_2010[c("mean_anom", "surface")]

Combine into one stars object
combined <- c(summary_2009_sel, summary_2010_sel)

Inspect
combined
...

Explanation

- **Attribute selection****: `summary_2009[c("mean_anom", "surface")]` keeps only those two attributes.
- **Combine with c()****: merges the two stars objects into one, provided they share the same grid, bounding box, resolution, and CRS.
- The result is a single stars object with four attributes: `mean_anom.1`, `surface.1` (from 2009) and `mean_anom.2`, `surface.2` (from 2010). You can rename them if needed.

```

This way, your final object contains only the **monthly mean anomaly** and **surface class** attributes for both years, while preserving the original raster dimensions and spatial metadata.

Would you like me to show how to **rename** the attributes clearly as 2009 vs 2010 so the combined object is easier to interpret?

### Message 59 - jia

if we want to use `c()` with `along = 3` to combine the two monthly objects along a new third dimension.  
how can i do this while keeping the existing x and y dimensions unchanged?

### Message 60 - Copilot

To combine your two monthly summary objects along a new **third dimension** (e.g. "year"), you can use `c(..., along = 3)` in `stars`. This keeps the existing ``x`` and ``y`` dimensions unchanged, while stacking the attributes into a new dimension.

Here's how you can do it:

```
```\nlibrary(stars)\n\n# Select only mean_anom and surface from each summary\nsummary_2009_sel <- summary_2009[c("mean_anom", "surface")]\nsummary_2010_sel <- summary_2010[c("mean_anom", "surface")]\n\n# Combine along a new dimension (3rd axis)\ncombined <- c(summary_2009_sel, summary_2010_sel, along = 3)\n\n# Inspect the dimensions\nst_dimensions(combined)\n```\n\n### Explanation\n\n- Attribute selection: Keeps only `mean_anom` and `surface` from each year.\n- c(..., along = 3): Stacks the two stars objects along a new dimension (the 3rd one, since `x` and `y` are the first two).\n- The result has dimensions: `x`, `y`, and a new dimension (often called `new_dim` or `band`).  
- Spatial metadata: Bounding box, resolution, and CRS remain unchanged because both inputs share the same grid.
```

Now you have one stars object with ``x`` and ``y`` intact, and a new dimension that separates December 2009 and December 2010.

Would you like me to show how to **label** the new dimension as "year" so it's easier to distinguish between 2009 and 2010 inside the combined object?

Message 61 - jia

after combining them, the third dimension needs to be year and contain the values 2009 and 2010
how can we use `st_set_dimensions()` to assign the dimension name and values?

Message 62 - Copilot

You can rename and assign values to the new third dimension using `st_set_dimensions()`. After combining with `c(..., along = 3)`, the third dimension will usually be called something like ``new_dim``. You can then set its name to `"year"` and assign the values ``2009`` and ``2010``.

Here's how:

```
```\nlibrary(stars)\n\n# Combine along new dimension\ncombined <- c(summary_2009[c("mean_anom", "surface")],\n              summary_2010[c("mean_anom", "surface")],\n              along = 3)\n\n# Rename the third dimension to "year" and assign values\ncombined <- st_set_dimensions(\n  combined,\n  which = 3,          # the third dimension\n  names = "year",      # new dimension name\n  values = c(2009, 2010) # values for that dimension\n)\n\n# Inspect\nst_dimensions(combined)\n```\n\n### Explanation\n\n- c(..., along = 3): stacks the two monthly summaries along a new third axis.\n- st_set_dimensions(): lets you rename and assign values to that axis.\n- which = 3: selects the third dimension (after `x` and `y`).
```

```
- `names = "year"`: renames the dimension.
- `values = c(2009, 2010)`: assigns the year labels to the slices.
```

Result: your final stars object has dimensions `x`, `y`, and `year`, with values `2009` and `2010`, and attributes `mean\_anom` and `surface`.

Would you like me to show how to **plot anomaly maps faceted by year** using this combined object?

### Message 63 - You

```
for "## 3.6 Derive Surface Classes"
ive edited your suggestions to fit my usecase
but it seems like i dont have enough coffee
"add_surface_class <- function(monthly) {
 surface_values <- case_when(
 monthly$land ~ "land",
 monthly$mean_ice >= 0.15 | is.na(monthly$mean_anom) ~ "sea ice",
 TRUE ~ "open ocean"
)
 surface_values <- factor(
 surface_values,
 levels = c("land", "sea ice", "open ocean")
)
 dim(surface_values) <- dim(monthly$land)
 monthly$surface <- surface_values
 monthly
}
summary_2009 <- add_surface_class(summary_2009)
summary_2010 <- add_surface_class(summary_2010)"
err -> "Error in `case_when()`:
! `..1 (left)` must be a logical vector, not a logical matrix.
Backtrace:

1. | global add_surface_class(summary_2009)
2. | | dplyr::case_when(...)
3. | | vctrs::vec_case_when(...)
4. | rlang::abort(message = message, call = call) at vctrs/R/case-when.R:188:3
"
how do you recon i fix this issue?
```

### Message 64 - Copilot

The error comes from how `case_when()` works: it expects **vectors**, but in your function you're passing entire stars arrays (matrices). `monthly$land` and `monthly$mean_ice` are 2D arrays, so `case_when()` complains because it doesn't know how to handle a matrix directly.

You can fix this by using **vectorized logical indexing** instead of `case_when()`. Here's a clean way to rewrite your function:

```
...
add_surface_class <- function(monthly) {
 # start with everything as "open ocean"
 surface_values <- rep("open ocean", length(monthly$land))

 # land first
 surface_values[monthly$land] <- "land"

 # sea ice: mean_ice >= 0.15 OR anomaly missing
 surface_values[!monthly$land &
 (monthly$mean_ice >= 0.15 | is.na(monthly$mean_anom))] <- "sea ice"

 # turn into factor with correct levels
 surface_values <- factor(surface_values,
 levels = c("land", "sea ice", "open ocean"))

 # reshape back to the same dimensions as land
 dim(surface_values) <- dim(monthly$land)

 # add as new attribute
 monthly$surface <- surface_values
 monthly
}

Apply to both summaries
summary_2009 <- add_surface_class(summary_2009)
summary_2010 <- add_surface_class(summary_2010)
...
```

### ### Why this works

- **Vector indexing**: Instead of `case_when()`, you assign values directly to subsets of the array.
- **Order of conditions**: Land first, then sea ice, then default open ocean.
- **Factor conversion**: Ensures the levels are ordered as required.
- **Dimension reset**: Keeps the raster grid intact.

This avoids the “logical matrix” error and gives you a proper categorical raster layer for surface classes.

Would you like me to also show how to **plot the surface factor with a custom palette** so land, sea ice, and open ocean are visually distinct?

### Message 65 - You

```
No missing surface classes
any(is.na(summary_2009[[1]]))
any(is.na(summary_2010[[1]]))
Every open ocean cell has a nonmissing anomaly
all(!is.na(combined_2009$mean_anom[summary_2009[[1]] == "open ocean"]))
all(!is.na(combined_2010$mean_anom[summary_2010[[1]] == "open ocean"]))
Every open ocean cell has mean ice below 0.15
all(combined_2009$mean_ice[summary_2009[[1]] == "open ocean"] < 0.15)
all(combined_2010$mean_ice[summary_2010[[1]] == "open ocean"] < 0.15)
Every cell with mean ice ≥ 0.15 is sea ice
all(summary_2009[[1]][combined_2009$mean_ice >= 0.15] == "sea ice")
all(summary_2010[[1]][combined_2010$mean_ice >= 0.15] == "sea ice")
update the verifier task to the updated variable
```

### Message 66 - Copilot

Right now your verification code is still pointing at `summary_2009[[1]]` and `summary_2010[[1]]`, which just grab the **first attribute** in each stars object. Since you’ve added a `surface` attribute, you should explicitly reference that instead. Likewise, the anomaly and ice checks should use the updated `summary_2009` / `summary_2010` objects rather than the older `combined_2009` / `combined_2010`.

Here’s the updated verifier task:

```
...
No missing surface classes
any(is.na(summary_2009$surface))
any(is.na(summary_2010$surface))

Every open ocean cell has a nonmissing anomaly
all(!is.na(summary_2009$mean_anom[summary_2009$surface == "open ocean"]))
all(!is.na(summary_2010$mean_anom[summary_2010$surface == "open ocean"]))

Every open ocean cell has mean ice below 0.15
all(summary_2009$mean_ice[summary_2009$surface == "open ocean"] < 0.15)
all(summary_2010$mean_ice[summary_2010$surface == "open ocean"] < 0.15)

Every cell with mean ice ≥ 0.15 is sea ice
all(summary_2009$surface[summary_2009$mean_ice >= 0.15] == "sea ice")
all(summary_2010$surface[summary_2010$mean_ice >= 0.15] == "sea ice")
...
```

### ### Key fixes

- **Explicit attribute selection**: use `$surface` instead of `[[1]]`.
- **Consistent object references**: use `summary_2009` / `summary_2010` for anomaly and ice checks.
- **Logical indexing**: ensures conditions are applied only to the relevant surface class cells.

This way, your checks directly validate the updated attributes (`surface`, `mean_anom`, `mean_ice`) inside the monthly summary objects.

Would you like me to show how to **wrap these checks into a reusable validation function** so you can run them on any monthly summary object in one call?

### Message 67 - jia

now the combined object should have dimensions 1440 by 720 by 2, with dimension names x, y, and year.  
how can i verify that the year values are 2009 and 2010 and that the bounding box, resolution, and crs are still correct?



## Message 68 - Copilot

To verify that your combined object has the correct **year values** and that the **spatial metadata** is unchanged, you can use ``st_dimensions()``, ``st_bbox()``, and ``st_crs()``:

```
...
library(stars)

Inspect dimensions
st_dimensions(combined)

Check year values
st_dimensions(combined)$year$values
Should return: 2009 2010

Check bounding box
st_bbox(combined)

Check resolution
st_dimensions(combined)xdelta
st_dimensions(combined)ydelta

Check CRS
st_crs(combined)
...

What each check does

- st_dimensions()****: confirms the object has dimensions `x`, `y`, and `year`, with sizes
 `1440 x 720 x 2`.
- Year values****: st_dimensions(combined)$year$values` should show `2009` and `2010`.
- st_bbox()****: verifies the bounding box (longitude/latitude extent) is unchanged.
- Resolution****: `delta` values for `x` and `y` should match your original grid spacing.
- st_crs()****: confirms the coordinate reference system (likely EPSG:4326) is still
 correct.
```

This way you can be confident that the combined object has the expected **1440 x 720 x 2 grid**, the **year dimension values 2009 and 2010**, and the same bounding box, resolution, and CRS as your original monthly summaries.

Would you like me to show how to **facet plots by year dimension** so you can visualize anomaly or surface classes side by side for 2009 vs 2010?

## Message 69 - You

```
"# No missing surface classes
any(is.na(summary_2009$surface))
any(is.na(summary_2010$surface))
Every open ocean cell has a nonmissing anomaly
all(!is.na(summary_2009$mean_anom[summary_2009$surface == "open ocean"]))
all(!is.na(summary_2010$mean_anom[summary_2010$surface == "open ocean"]))
Every open ocean cell has mean ice below 0.15
all(summary_2009$mean_ice[summary_2009$surface == "open ocean"] < 0.15)
all(summary_2010$mean_ice[summary_2010$surface == "open ocean"] < 0.15)
Every cell with mean ice ≥ 0.15 is sea ice
all(summary_2009$surface[summary_2009$mean_ice >= 0.15] == "sea ice")
all(summary_2010$surface[summary_2010$mean_ice >= 0.15] == "sea ice")" -> i realised that this
logic is not enough to cover the checks or missing compounded logics
"1. All surface values belong to one of the three classes: "land", "sea ice", or "open
ocean". No surface value is missing.
2. Every cell with mean_ice >= 0.15 is classified as "sea ice", as is every non-land
cell with a missing mean_anom.
3. Every "open ocean" cell has a nonmissing mean_anom and mean_ice < 0.15.
" -> this is the req that i need
```

## Message 70 - Copilot

Got it – you want your verification logic to directly reflect the **three requirements** you listed. Instead of piecemeal checks, you can write them as clear assertions against the updated ``summary_2009`` and ``summary_2010`` objects:

```
...

1. All surface values are valid and non-missing
all(summary_2009$surface %in% c("land", "sea ice", "open ocean"))
all(summary_2010$surface %in% c("land", "sea ice", "open ocean"))

2. Every cell with mean_ice >= 0.15 is "sea ice",
and every non-land cell with missing mean_anom is "sea ice"
all(summary_2009$surface[summary_2009$mean_ice >= 0.15] == "sea ice")
```

```

all(summary_2010$surface[summary_2010$mean_ice >= 0.15] == "sea ice")

all(summary_2009$surface[!summary_2009$land & is.na(summary_2009$mean_anom)] == "sea ice")
all(summary_2010$surface[!summary_2010$land & is.na(summary_2010$mean_anom)] == "sea ice")

3. Every "open ocean" cell has nonmissing mean_anom and mean_ice < 0.15
all(!is.na(summary_2009$mean_anom[summary_2009$surface == "open ocean"]))
all(!is.na(summary_2010$mean_anom[summary_2010$surface == "open ocean"]))

all(summary_2009$mean_ice[summary_2009$surface == "open ocean"] < 0.15)
all(summary_2010$mean_ice[summary_2010$surface == "open ocean"] < 0.15)
...

Why this covers everything

- **Surface validity****: ensures no missing or unexpected class values.
- **Sea ice rules****: enforces both the ≥0.15 ice threshold and the missing anomaly condition
 for non-land cells.
- **Open ocean rules****: guarantees anomaly is present and ice is below threshold.

 With these checks, you now have a complete verifier that matches your three requirements
 exactly.

 Would you like me to show how to **wrap these into a single reusable function** so you can run
 all checks at once for any monthly summary object?

```

## Message 71 - You

```

"add_surface_class <- function(monthly) {
start with everything as "open ocean"
surface_values <- rep("open ocean", length(monthly$land))
land first
surface_values[monthly$land] <- "land"
sea ice: mean_ice >= 0.15 OR anomaly missing
surface_values[!monthly$land &
(monthly$mean_ice >= 0.15 | is.na(monthly$mean_anom))] <- "sea ice"
turn into factor with correct levels
surface_values <- factor(surface_values,
levels = c("land", "sea ice", "open ocean"))
reshape back to the same dimensions as land
dim(surface_values) <- dim(monthly$land)
add as new attribute
monthly$surface <- surface_values
monthly
}
Apply to both summaries
summary_2009 <- add_surface_class(summary_2009)
summary_2010 <- add_surface_class(summary_2010)
" -> " Its dimensions are named x, y, and year and have lengths 1440, 720, and 2, respectively.
2. The st_get_dimension_values() function returns 2009 and 2010 for the year dimension.
3. It contains exactly the attributes mean_anom and surface.
4. It has the same bounding box, spatial resolution, and CRS as both summary_2009
and summary_2010." we need to verify the code

```

## Message 72 - Copilot

To verify that your \*\*combined stars object\*\* meets the four requirements, you can use the following checks:

```

...

library(stars)

1. Dimensions are named x, y, and year with lengths 1440, 720, and 2
st_dimensions(combined)

2. Year values are 2009 and 2010
st_get_dimension_values(combined, "year")
Expected: 2009 2010

3. Attributes are exactly mean_anom and surface
names(combined)
Expected: "mean_anom" "surface"

4. Bounding box, resolution, and CRS match the originals
st_bbox(combined)
st_dimensions(combined)xdelta
st_dimensions(combined)ydelta
st_crs(combined)

```

```
...
```

### ### Explanation

```
- **st_dimensions()**: confirms the object has dimensions `x`, `y`, and `year` with sizes `1440`
`x 720 x 2`.
- **st_get_dimension_values()**: retrieves the values for the `year` dimension, which should be
`2009` and `2010`.
- **names()**: ensures only `mean_anom` and `surface` are present.
- **st_bbox()**, **delta values**, and **st_crs()**: confirm the bounding box, resolution, and
CRS are unchanged compared to your original summary objects.
```

Running these checks will give you confidence that the combined object is correctly structured, has the right year labels, contains only the required attributes, and retains the original spatial metadata.

Would you like me to show how to **plot** the combined object faceted by year so you can visually confirm the integrity of the data for 2009 and 2010?

## Message 73 - You

lets work on 3.9

```
this is my 3.8 plot code -> ```{r}
```

```
#| label: draft-faceted-map
```

```
#| fig-width: 10
```

```
#| fig-height: 8
```

```
ggplot(dec_summaries) +
```

```
 geom_stars(aes(fill = mean_anom)) +
```

```
 facet_wrap(vars(year), ncol = 1)
```

```
```
```

currently the scale is unsuitable for sea-surface-temperature

anomalies because 0 °C has a specific meaning

observed temperature equals its 1971-2000

climatological value. The final scale should place this reference value at the scale's midpoint and

should distinguish negative from positive anomalies.

add an attribute named open_ocean_anom to dec_summaries

Set it equal to mean_anom

at cells classified as "open ocean" and to NA elsewhere. Keep mean_anom unchanged

Restricting

the mapped attribute to open-ocean cells ensures that anomalies hidden beneath the sea-ice layer do not influence the scale.

3.9 Design a Diverging Fill Scale

```
```{r}
```

```
#| label: create-open-ocean-mask
```

```
open_ocean_anom <- dec_summaries$mean_anom
```

```
open_ocean_anom[
```

```
 dec_summaries$surface != "open ocean"
```

```
] <- NA_real_
```

```
dec_summaries$open_ocean_anom <- open_ocean_anom
```

```
``` -> lets work from here
```

Message 74 - Copilot

Perfect – you've already masked anomalies to open ocean cells with `open\_ocean\_anom`. The next step is to design a **diverging fill scale** so that 0 °C sits at the midpoint, negatives are clearly distinguished from positives, and the map only reflects open ocean anomalies.

Here's how you can extend your plotting code:

```
...
```

```
## 3.9 Design a Diverging Fill Scale
```

```
#| label: faceted-open-ocean-map
```

```
#| fig-width: 10
```

```
#| fig-height: 8
```

```
library(ggplot2)
```

```
ggplot(dec_summaries) +
```

```
  geom_stars(aes(fill = open_ocean_anom)) +
```

```
  facet_wrap(vars(year), ncol = 1) +
```

```
  scale_fill_gradient2(
```

```
    name = "SST anomaly (°C)",
```

```
    midpoint = 0,
```

```
    low = "blue",           # 0 °C is the reference climatology
```

```
    mid = "white",          # negative anomalies
```

```
    high = "red",           # reference value
```

```
    # positive anomalies
```

```

    na.value = "grey90"      # sea ice and land masked
  ) +
  theme_minimal()
```

```

### Why this works

- **open\_ocean\_anom**: ensures anomalies under sea ice don't distort the scale.
- **scale\_fill\_gradient2**: places 0 °C at the midpoint, with symmetric colors for negative vs positive anomalies.
- **Color choices**: blue for colder than climatology, red for warmer, white for neutral.
- **NA handling**: land and sea ice cells show up in a neutral grey, keeping focus on open ocean.

This gives you a faceted map for 2009 and 2010 where the fill scale is scientifically meaningful: anomalies colder than climatology are clearly separated from warmer ones, and the reference climatology is visually centered.

Would you like me to also show how to **customize the legend breaks** (e.g. -2, -1, 0, +1, +2 °C) so the scale is easier to interpret at a glance?

## Message 75 - jia

now we need to derive one shared colour-scale limit from the data for both years.  
 from our understanding, we should take the absolute values of all nonmissing open\_ocean\_anom cells, calculate the 99th percentile using quantile(), and set na.rm = true.  
 how can i calculate this without manually setting the limit?

## Message 76 - Copilot

Exactly – you don't want to hard code the limits, you want to derive them from the data itself.  
 The workflow is:

1. Take all **non missing open ocean anomalies** across both years.
2. Compute their **absolute values**.
3. Use `quantile()` with `probs = 0.99` and `na.rm = TRUE` to get the 99th percentile.
4. Use that value as the symmetric limit for your diverging scale.

Here's how you can do it in R:

```

```
## 3.9 Shared Colour-Scale Limit

# Flatten both years' anomalies into one vector
anom_values <- c(dec_summaries$open_ocean_anom)

# Compute 99th percentile of absolute values
limit <- quantile(abs(anom_values), probs = 0.99, na.rm = TRUE)

limit
# Example output: 2.3 (°C)
```

```

Then plug `limit` into your plotting scale:

```

```
ggplot(dec_summaries) +
  geom_stars(aes(fill = open_ocean_anom)) +
  facet_wrap(vars(year), ncol = 1) +
  scale_fill_gradient2(
    name = "SST anomaly (°C)",
    midpoint = 0,
    low = "blue",
    mid = "white",
    high = "red",
    na.value = "grey90",
    limits = c(-limit, limit) # symmetric around 0
  ) +
  theme_minimal()
```

```

### Why this works

- **Shared scale**: ensures both years are plotted with the same limits, so anomalies are directly comparable.
- **99th percentile**: avoids extreme outliers from stretching the scale.
- **Symmetric limits**: places 0 °C at the midpoint, distinguishing negative vs positive anomalies clearly.

This way, your colour scale is data driven, robust to outliers, and scientifically meaningful.

Would you like me to also show how to **add labelled breaks at -limit, 0, and +limit** so the legend is easier to interpret?

### Message 77 - You

summarize this for me "3.9 Design a Diverging Fill Scale

The first map used ggplot2's default fill scale. This scale is unsuitable for sea-surface-temperature

anomalies because 0 °C has a specific meaning: The observed temperature equals its 1971-2000 climatological value. The final scale should place this reference value at the scale's midpoint and

should distinguish negative from positive anomalies.

First, add an attribute named open\_ocean\_anom to dec\_summaries. Set it equal to mean\_anom at cells classified as "open ocean" and to NA elsewhere. Keep mean\_anom unchanged. Restricting the mapped attribute to open-ocean cells ensures that anomalies hidden beneath the sea-ice layer do not influence the scale.

16

Inspect the distribution of abs(open\_ocean\_anom) across both years. Set limit to the 99th percentile of its nonmissing values (the open-ocean cells of both panels combined), rounded upward to the next whole degree Celsius. Use this value for both panels.

Revise the map so that its fill scale has the following properties:

- It uses the ColorBrewer RdYlBu diverging palette, reversed so that cold (blue) colors mark negative anomalies and warm (red) colors mark positive ones.

- It has symmetric endpoints -limit and limit.

- It squishes values outside those limits to the nearest endpoint instead of treating them as missing—a cell whose anomaly exceeds limit, for instance, is drawn in the deep red of the limit endpoint rather than left blank.

Leave land and sea ice at the default fill for now; they will receive their own layer and colors in

the next section." -> simple point form of what i need to do

### Message 78 - Copilot

Here's a simple point form summary of what you need to do for **3.9 Design a Diverging Fill Scale**:

- **Mask anomalies**:

Add `open\_ocean\_anom` to `dec\_summaries`, equal to `mean\_anom` only for `"open ocean"` cells, `NA` elsewhere.

- **Derive shared limit**:

- Collect all non missing `open\_ocean\_anom` values across both years.

- Take absolute values.

- Compute the 99th percentile with `quantile(..., probs = 0.99, na.rm = TRUE)`.

- Round up to the next whole °C.

- Use this as the symmetric limit for both panels.

- **Design diverging scale**:

- Use ColorBrewer **RdYlBu** palette, reversed (blue = negative, red = positive).

- Set symmetric endpoints `-limit` and `+limit`.

- Squish values outside limits to the nearest endpoint (so extreme anomalies are shown at the max color, not blank).

- Keep land and sea ice at default fill for now.

That's the streamlined checklist: mask → compute limit → apply diverging scale.

Would you like me to show the exact **ggplot2** code snippet that implements all three steps together, including the `scale_fill_gradientn()` with RdYlBu reversed?

### Message 79 - You

Mask anomalies to open ocean

### Message 80 - Copilot

To **mask anomalies to open ocean**, you simply create a new attribute that copies `mean\_anom` but sets all non-open ocean cells to `NA`. This way, only anomalies from open ocean contribute to the color scale.

Here's the code:

```
```
```

```
## 3.9 Create Open-Ocean Mask
```

```
open_ocean_anom <- dec_summaries$mean_anom
```

```
# Replace values outside open ocean with NA
```

```
open_ocean_anom[dec_summaries$surface != "open ocean"] <- NA_real_
```

```
# Add as new attribute
dec_summaries$open_ocean_anom <- open_ocean_anom
````
```

### Why this step matters

- **Mask anomalies**: ensures anomalies hidden under sea ice don't distort the scale.
- **Keep mean\_anom unchanged**: you still retain the full anomaly field for other uses.
- **Focus on open ocean**: the diverging scale will now only reflect anomalies where they are meaningful.

With this mask in place, you can safely design a diverging fill scale centered at 0 °C, knowing that only open ocean anomalies drive the limits.

Would you like me to show how to **compute the shared 99th percentile limit** from this new ``open_ocean_anom`` attribute?

## Message 81 - jia

now we want to draw the open-ocean anomaly map using:  
geom\_stars() with open\_ocean\_anom mapped to fill  
scale\_fill\_distiller()  
a diverging RdYlBu palette  
direction = -1  
limits from -limit to limit  
oob = squish  
the two years stacked in one column  
how can i put this together?

## Message 82 - Copilot

You can put all the pieces together into one ggplot call that uses your masked anomalies, a diverging **RdYlBu** palette, symmetric limits, and squishing for out of range values. Here's the full code:

```
````
```

3.9 Open-Ocean Anomaly Map

```
#| label: faceted-open-ocean-map
#| fig-width: 10
#| fig-height: 8
```

```
library(ggplot2)
library(scales) # for squish
```

```
ggplot(dec_summaries) +
  geom_stars(aes(fill = open_ocean_anom)) +
  facet_wrap(vars(year), ncol = 1) +
  scale_fill_distiller(
    name = "SST anomaly (°C)",
    type = "div",           # diverging palette
    palette = "RdYlBu",     # ColorBrewer RdYlBu
    direction = -1,         # reverse so blue = negative, red = positive
    limits = c(-limit, limit), # symmetric endpoints
    oob = squish,           # squish values outside limits
    na.value = "grey90"     # land/sea ice left neutral
  ) +
  theme_minimal()
````
```

### Key points

- **geom\_stars**: maps ``open_ocean_anom`` to fill.
- **scale\_fill\_distiller**: uses a diverging ColorBrewer palette.
- **direction = -1**: flips the palette so cold anomalies are blue, warm anomalies red.
- **limits = c(-limit, limit)**: symmetric around zero, derived from your 99th percentile calculation.
- **oob = squish**: extreme values are shown at the endpoint color instead of blank.
- **facet\_wrap(vars(year), ncol = 1)**: stacks the two years vertically in one column.

This produces a faceted map where both years share the same diverging scale, centered at 0 °C, with clear visual distinction between negative and positive anomalies.

Would you like me to also show how to **add custom legend breaks** (e.g. -2, -1, 0, +1, +2 °C) so the scale is easier to interpret?

## Message 83 - You

## 3.8 Draft a First Faceted Raster Map

```
```{r}
#| label: draft-faceted-map
#| fig-width: 10
#| fig-height: 8
dec_summaries <- combined
ggplot(dec_summaries) +
  geom_stars(aes(fill = mean_anom)) +
  facet_wrap(vars(year), ncol = 1)
```
```

Error in `fortify()`:

! `data` must be a <data.frame>, or an object coercible by `fortify()`, or a valid <data.frame>-like object coercible by `as.data.frame()`.

Caused by error in `check\_data\_frame\_like()`:

! `dim(data)` must return an <integer> of length 2.

## Message 84 - Copilot

That error happens because `ggplot()` doesn't know how to handle your `stars` object directly unless it's in the right format. The fix is to make sure you're using `geom\_stars()` **from the stars package** and that your `combined` object is indeed a `stars` object with dimensions `x`, `y`, and `year`.

Here's how you can put it together cleanly:

```
```
## 3.8 Draft a First Faceted Raster Map

#| label: draft-faceted-map
#| fig-width: 10
#| fig-height: 8

library(ggplot2)
library(stars)

# Ensure dec_summaries is a stars object
dec_summaries <- combined

ggplot() +
  geom_stars(data = dec_summaries, aes(fill = mean_anom)) +
  facet_wrap(~year, ncol = 1)
```
```

### ### Key adjustments

- **geom\_stars**: pass the stars object via `data = dec\_summaries`.
- **Facet formula**: use `~year` instead of `vars(year)` – this works directly with the dimension values.
- **Check object type**: run `class(combined)`; it should include `"stars"`. If not, convert with `st\_as\_stars(combined)`.

With these changes, ggplot will correctly facet your raster map by year, stacking the two panels in one column.

Would you like me to also show how to **add the diverging fill scale** we discussed earlier so the anomalies are centered at 0 °C?

## Message 85 - jia

ow we want the facet labels to show both the year and the climate phase instead of only showing 2009 and 2010.

our plan is to create a named character vector where 2009 maps to December 2009 (El Niño) and 2010 maps to December 2010 (La Niña).

how can i define this vector so it can later be used with labeller()?

## Message 86 - Copilot

You can define a simple **named character vector** that maps the numeric year values to the labels you want. This vector can then be passed to `labeller()` in your facet call.

Here's how to set it up:

```
```
# Define mapping from year values to labels
year_labels <- c(
  "2009" = "December 2009 (El Niño)",
  "2010" = "December 2010 (La Niña)"
)
```

```
# Example usage with labeller
ggplot(dec_summaries) +
  geom_stars(aes(fill = open_ocean_anom)) +
  facet_wrap(~year, ncol = 1, labeller = labeller(year = year_labels))
...
```

Why this works

- **Named vector mapping**: the names (`"2009"`, `"2010"`) must match the values in your `year`` dimension.
- **Custom labels**: the values are the strings you want displayed in the facet strip.
- **labeller()**: applies the mapping when drawing facets.

With this, your facet labels will show **"December 2009 (El Niño)"** and **"December 2010 (La Niña)"** instead of just the raw year numbers.

Would you like me to also show how to **add a title and caption** to the plot so the climate phases are explained in the figure itself?

Message 87 - jia

for the surface layer, we want:

land as grey70

sea ice as white

open ocean as transparent

how can i store these as a named vector called `surface_colours` so that the names match the factor values in `surface`?

Message 88 - Copilot

You can store the surface layer colours as a **named character vector**, where the names match the factor levels in your `surface`` attribute (`"land"`, `"sea ice"`, `"open ocean"`). That way, `ggplot` will automatically match the right fill colour to each class.

Here's how to define it:

```
...
# Define surface colours
surface_colours <- c(
  "land"      = "grey70",
  "sea ice"   = "white",
  "open ocean" = "transparent"
)
```

```
surface_colours
...
```

Why this works

- **Named vector mapping**: the names must exactly match the factor values in `surface``.
- **Colour assignment**: `ggplot` will use this vector when you call `scale_fill_manual(values = surface_colours)``.
- **Transparency for open ocean**: ensures the anomaly layer beneath is visible, while land and sea ice are drawn with their own colours.

With this setup, you can later add the surface layer using `geom_stars(aes(fill = surface)) + scale_fill_manual(values = surface_colours)``, and it will render land in grey, sea ice in white, and leave open ocean transparent.

Would you like me to also show how to **overlay the surface layer on top of the anomaly map** so the two layers combine correctly?

Message 89 - jia

and we want only land and sea ice to appear in the surface legend because open ocean is already represented through the anomaly colours.

how can i create a `surface_breaks` vector for these two categories and separate labels where sea ice is shown as Sea-ice concentration $\geq 15\%$?

Message 90 - Copilot

You can define both the **breaks vector** (which categories to show in the legend) and a separate **labels vector** (how they should appear) so they match the factor values in your `surface`` attribute:

```
...
# Only land and sea ice should appear in the legend
surface_breaks <- c("land", "sea ice")
```



```
# Custom labels for those categories
surface_labels <- c(
  "land" = "Land",
  "sea ice" = "Sea-ice concentration ≥ 15%"
)

surface_breaks
surface_labels
```


How to use them

- **surface_breaks**: tells ggplot which factor levels to include in the legend.
- **surface_labels**: provides custom text for those breaks.
- Together with your surface_colours vector, you can apply them in scale_fill_manual():


```
```
ggplot(dec_summaries) +
 geom_stars(aes(fill = surface)) +
 scale_fill_manual(
 values = surface_colours,
 breaks = surface_breaks,
 labels = surface_labels
)
```
```


```

This way, the legend will only show **\*\*Land\*\*** and **\*\*Sea ice concentration ≥ 15%\*\***, while open ocean remains transparent and excluded from the legend.

Would you like me to also show how to **\*\*overlay this surface layer on top of your anomaly map\*\*** so the two layers combine seamlessly?

## Message 91 - jia

ok now we want to build the map in layers.  
 the first layer should use `geom_stars()` with `open_ocean_anom` mapped to fill, followed by the same diverging scale from part 3.9.  
 this time we also want missing anomaly values to be transparent.  
 how can i add `na.value = "transparent"` to the anomaly scale?

## Message 92 - Copilot

You can add `na.value = "transparent"` directly inside your anomaly scale definition. That way, any missing anomaly values will be drawn as transparent instead of a solid colour. Here's how the layered map code looks:

```
```
## 3.10 Build Map in Layers

#| label: layered-open-ocean-map
#| fig-width: 10
#| fig-height: 8

library(ggplot2)
library(scales)

ggplot(dec_summaries) +
  # First layer: anomalies
  geom_stars(aes(fill = open_ocean_anom)) +
  facet_wrap(~year, ncol = 1) +
  scale_fill_distiller(
    name = "SST anomaly (°C)",
    type = "div",
    palette = "RdYlBu",
    direction = -1, # blue = negative, red = positive
    limits = c(-limit, limit), # symmetric endpoints
    oob = squish, # squish out-of-range values
    na.value = "transparent" # missing anomalies drawn transparent
  )
```
```

### ### Key points

- **\*\*geom\_stars\*\***: maps `open_ocean_anom` to fill.
- **\*\*scale\_fill\_distiller\*\***: diverging `RdYlBu` palette, reversed.
- `na.value = "transparent"`: ensures missing anomalies don't add unwanted colour patches.
- **\*\*limits\*\***: symmetric around zero, derived from your 99th percentile.
- **\*\*oob = squish\*\***: extreme values are capped at the endpoint colours.

This gives you a clean anomaly layer where only valid open ocean anomalies are coloured, missing values are transparent, and the scale is scientifically meaningful.

Would you like me to also show how to **overlay the surface layer with its own colours** on top of this anomaly layer so land and sea ice are visible?

### Message 93 - jia

both the anomaly layer and the surface layer use the fill aesthetic, but one is continuous and the other is categorical.

from our understanding, we need `ggnewscale::new_scale_fill()` between the two layers so the second layer can use a separate fill scale.

is this correct, and where should `new_scale_fill()` be placed?

### Message 94 - Copilot

Yes – your understanding is correct. When you have **two layers that both use ``fill``**, but one is continuous (``open_ocean_anom``) and the other categorical (``surface``), you need to reset the fill scale in between. That's exactly what ``ggnewscale::new_scale_fill()`` is for.

#### ### Placement

You insert ``new_scale_fill()`` **after the anomaly layer and its scale**, but **before** adding the surface layer. The order looks like this:

```
...
library(ggplot2)
library(ggnewscale)

ggplot(dec_summaries) +
 # First layer: anomalies
 geom_stars(aes(fill = open_ocean_anom)) +
 facet_wrap(~year, ncol = 1) +
 scale_fill_distiller(
 name = "SST anomaly (°C)",
 type = "div",
 palette = "RdYlBu",
 direction = -1,
 limits = c(-limit, limit),
 oob = squish,
 na.value = "transparent"
) +

 # Reset fill scale for the next layer
 new_scale_fill() +

 # Second layer: surface classes
 geom_stars(aes(fill = surface)) +
 scale_fill_manual(
 values = surface_colours,
 breaks = surface_breaks,
 labels = surface_labels,
 name = NULL
)
...
```

#### ### Why this works

- **`new_scale_fill`**: resets the fill aesthetic so the second layer can have its own discrete scale.
- **First scale**: continuous diverging palette for anomalies.
- **Second scale**: categorical manual colours for land/sea ice.
- **Transparent open ocean**: ensures anomaly colours show through.

With this structure, both layers render correctly: anomalies with a diverging continuous scale, and land/sea ice with a categorical legend.

Would you like me to also show how to **combine the two legends into a cleaner layout** so the anomaly legend and surface legend don't overlap awkwardly?

### Message 95 - jia

after `new_scale_fill()`, we want to add another `geom_stars()` layer using `surface` as the fill variable.

how can i then apply `scale_fill_manual()` using:

```
surface_colours
surface_breaks
surface_labels
```

```
drop = false
transparent missing values?
```

## Message 96 - Copilot

Exactly – after you reset the fill scale with `ggnewscale::new\_scale\_fill()`, you can add your **\*\*surface layer\*\*** and then apply a discrete scale with `scale\_fill\_manual()`. Here's how to put it together:

```
...

library(ggplot2)
library(ggnewscale)

ggplot(dec_summaries) +
 # First layer: anomalies
 geom_stars(aes(fill = open_ocean_anom)) +
 facet_wrap(~year, ncol = 1) +
 scale_fill_distiller(
 name = "SST anomaly (°C)",
 type = "div",
 palette = "RdYlBu",
 direction = -1,
 limits = c(-limit, limit),
 oob = squish,
 na.value = "transparent" # anomalies missing → transparent
) +

 # Reset fill scale
 new_scale_fill() +

 # Second layer: surface classes
 geom_stars(aes(fill = surface)) +
 scale_fill_manual(
 values = surface_colours, # your named colours vector
 breaks = surface_breaks, # only land & sea ice
 labels = surface_labels, # custom legend labels
 drop = FALSE, # keep all factor levels
 na.value = "transparent" # missing surface values → transparent
)
...

Key points

- **new_scale_fill****: resets the fill aesthetic so the second layer can have its own discrete scale.
- **scale_fill_manual****: applies your `surface_colours`, `surface_breaks`, and `surface_labels`.
- `drop = FALSE`: ensures unused factor levels aren't dropped.
- `na.value = "transparent"`: makes missing surface values invisible, so only land and sea ice are shown.

This layering gives you a clean anomaly map underneath, with land and sea ice drawn on top in their own colours and legend.

Would you like me to also show how to **arrange the two legends side by side** so the anomaly and surface legends don't stack awkwardly?
```

## Message 97 - You

## 3.10 Add Land and Sea Ice as Surface Layers

```
... {r}
Define mapping from year values to labels
year_labels <- c(
 "2009" = "December 2009 (El Niño)",
 "2010" = "December 2010 (La Niña)"
)
Define surface colours
surface_colours <- c(
 "land" = "grey70",
 "sea ice" = "white",
 "open ocean" = "transparent"
)
surface_colours
...
... {r}
Only land and sea ice should appear in the legend
surface_breaks <- c("land", "sea ice")
Custom labels for those categories
```

```

surface_labels <- c(
 "land" = "Land",
 "sea ice" = "Sea-ice concentration ≥ 15%"
)
surface_breaks
surface_labels
...

3.10.1 Plot
``` {r}
ggplot() +
  geom_stars(data = dec_summaries, aes(fill = mean_anom)) +
  scale_fill_manual(
    values = surface_colours,
    breaks = surface_breaks,
    labels = surface_labels
  )
...

```

the plot sea ice is showing up as yellow, what is the root cause for this? even tho the code shows as white for sea ice?

Message 98 - Copilot

The reason your **sea ice** is showing up as yellow is that in your plotting code you're still mapping `mean_anom` to the fill aesthetic, but then applying `scale_fill_manual()` that was designed for the **categorical** `surface` attribute.

Here's the root cause:

- `geom_stars(aes(fill = mean_anom))` → this tells ggplot to treat the fill as a **continuous numeric variable**.
- `scale_fill_manual(values = surface_colours, ...)` → this expects a **factor with levels** like `"land"`, `"sea ice"`, `"open ocean"`.
- Because the scale and the data type don't match, ggplot falls back to default behaviour and assigns colours incorrectly (hence sea ice showing up as yellow from the continuous scale).

How to fix it

If you want to plot the **surface layer**, you need to map `surface` to fill instead of `mean_anom`:

```

...
ggplot() +
  geom_stars(data = dec_summaries, aes(fill = surface)) +
  scale_fill_manual(
    values = surface_colours,
    breaks = surface_breaks,
    labels = surface_labels,
    drop = FALSE,
    na.value = "transparent"
  )
...

```

Key takeaway

- **mean_anom** → continuous anomalies, should use a diverging scale.
- **surface** → categorical land/sea ice/open ocean, should use `scale_fill_manual()`.
- Mixing them up causes ggplot to misinterpret the mapping and apply the wrong colours.

Once you switch the fill mapping to `surface`, sea ice will correctly appear as white, land as grey70, and open ocean as transparent.

Would you like me to also show how to **layer** the anomaly map underneath the surface layer so both render together with separate fill scales?

Message 99 - You

3.11 Compare Equal-Area World Projections

The map is still drawn directly in longitude and latitude. This equirectangular view stretches high-latitude regions far wider than equatorial regions of equal true area. Because the map invites comparisons of the spatial extent of temperature anomalies and sea ice, redraw it with an

equal-area projection. Compare two alternatives:

17

Projection CRS definition

Equal Earth +proj=eqearth +lon_0=180 +ellps=WGS84 +R_A +units=m +no_defs
+type=crs

Mollweide +proj=moll +lon_0=180 +ellps=WGS84 +R_A +units=m +no_defs
+type=crs

Both projections use 180° as the central meridian, which places the Pacific Ocean in the center of each panel and aligns the map's left and right edges with the raster's existing $0^\circ/360^\circ$ longitude boundary.

Reprojecting a raster means resampling it: `st_warp()` lays the cube onto a grid defined in the target CRS, and `coord_equal()` then draws that already-projected grid. With your chosen projection stored as `crs` (one of the two definitions above, read with `st_crs()`), trim the poles, build the target grid explicitly, and resample:

```
# Trim the poles (outside Mollweide's domain) and build the target grid
# explicitly, or st_warp() would cover only half of this global source.
keep <- which(abs(st_get_dimension_values(dec_summaries, "y")) <= 89.5)
trimmed <- dec_summaries[, , keep, ]
target <- st_as_stars(
  st_bbox(st_transform(trimmed, crs)),
  dx = 50000, # 50-km cells (CRS units are meters)
  dy = 50000
)
warped <- st_warp(trimmed, target, method = "near")
```

Nearest-neighbor resampling (method = "near") keeps surface a factor and does not blur the anomaly across the coastline. Equal Earth adds one artifact: Its inverse projection wraps data into the rectangle's corners, which lie outside the round map. Crop the warped grid to the projected world boundary to blank them. Build that boundary as a longitude-latitude rectangle spanning the trimmed latitudes, with its left and right edges a hair inside 0° and 360° . Densify its edges with `st_segmentize()`, giving the maximum segment length in degrees, so that the transformed rectangle follows the projection's curve, and then transform it to the target CRS. The same boundary traces the frame that Section 3.12 asks for.

Because the two projections differ only in their CRS, wrap the steps above in one function of the CRS and call it once per projection. Draw each map with the two fill layers and scales from Section 3.10, stack the two Decembers in one column, and replace the default coordinate system with `coord_equal()`. With your team, compare the two maps and answer the following questions, using 2-3 sentences each:

- How does each projection represent the polar regions?
- How clearly does each projection show the equatorial Pacific?
- Which projection uses the rectangular panel space more effectively?
- Which projection makes the contrast between the two Decembers more visually prominent?

Choose one projection for the final map and justify your choice in 3-5 sentences." -> summarize this for me in point form

Message 100 - Copilot

Here's a clear point form summary of **3.11 Compare Equal Area World Projections**:

- **Problem**: Current map uses longitude/latitude (equirectangular), which distorts high latitudes compared to equatorial regions.
- **Goal**: Redraw with an equal area projection to fairly compare anomaly and sea ice extent.
- **Projections**:
 - Equal Earth (`+proj=eqearth +lon_0=180 ...`)
 - Mollweide (`+proj=moll +lon_0=180 ...`)
 - Both centered at 180° longitude (Pacific in the middle).
- **Resampling**:
 - Trim poles to $\pm 89.5^\circ$.
 - Build target grid with 50 km cells (`dx = dy = 50000`).
 - Use `st_warp(..., method = "near")` for nearest neighbor resampling (preserves factor surface, avoids coastline blur).
- **Equal Earth artifact**: Inverse projection wraps data into corners outside the round map → crop to projected world boundary.
- **Boundary construction**:
 - Make lon lat rectangle spanning trimmed latitudes, edges just inside $0^\circ/360^\circ$.
 - Densify edges with `st_segmentize()` (max segment length in degrees).
 - Transform rectangle to target CRS → crop warped grid.
- **Reusable function**: Wrap trimming, grid building, warping, and cropping into one function parameterized by CRS.
- **Plotting**:
 - Draw maps with both fill layers (anomaly + surface).
 - Stack the two Decembers in one column.
 - Use `coord_equal()` for correct aspect ratio.
- **Comparison questions**:
 - How each projection represents polar regions.
 - Clarity of equatorial Pacific.
 - Efficiency of rectangular panel space.
 - Which projection makes December contrast more visually prominent.
- **Final choice**: Select one projection for the final map and justify in 3-5 sentences.

That's the streamlined checklist: distortions → equal area projections → resampling/cropping → reusable function → plotting → compare → choose.

Would you like me to also show a **code skeleton** for the reusable projection function so you can plug in either Equal Earth or Mollweide CRS directly?

Message 101 - You

lets work on the equal_earth_crs and mollweide_crs projection, how can i start?

Message 102 - Copilot

Great – the starting point is simply to **define the two CRS objects** using `st_crs()` with the projection strings you've been given. Once you have them, you can pass them into your warp/cropping function.

Here's how you can begin:

```
```  
library(sf)
library(stars)

Equal Earth projection (central meridian at 180°)
equal_earth_crs <- st_crs(
 "+proj=eqearth +lon_0=180 +ellps=WGS84 +R_A +units=m +no_defs +type=crs"
)

Mollweide projection (central meridian at 180°)
mollweide_crs <- st_crs(
 "+proj=moll +lon_0=180 +ellps=WGS84 +R_A +units=m +no_defs +type=crs"
)

equal_earth_crs
mollweide_crs
```  
  
### Next steps with these CRS objects  
  
- Trim poles: restrict latitudes to ±89.5° before warping.  
- Build target grid: use st_as_stars(st_bbox(st_transform(trimmed, crs)), dx = 50000, dy = 50000).  
- Warp: resample with st_warp(trimmed, target, method = "near").  
- Crop Equal Earth corners: build a lon lat rectangle, densify with st_segmentize(), transform to CRS, and crop.  
- Reusable function: wrap these steps so you can call once with equal_earth_crs and once with mollweide_crs.
```

With `equal_earth_crs` and `mollweide_crs` defined, you're ready to plug them into your warp/crop workflow and generate the two maps for comparison.

Would you like me to sketch out the **function skeleton** that takes a CRS as input and returns the warped/cropped stars object, so you can reuse it for both projections?

Message 103 - You

yupp lets move on to CRS objects

Message 104 - Copilot

Perfect – now that we're moving on to **CRS objects**, the first step is to define them explicitly so they can be passed into your warp/cropping workflow. You'll use `st_crs()` with the projection strings provided.

Here's how you can set them up:

```
```  
library(sf)
library(stars)

Equal Earth projection (central meridian at 180°)
equal_earth_crs <- st_crs(
 "+proj=eqearth +lon_0=180 +ellps=WGS84 +R_A +units=m +no_defs +type=crs"
)

Mollweide projection (central meridian at 180°)
mollweide_crs <- st_crs(
 "+proj=moll +lon_0=180 +ellps=WGS84 +R_A +units=m +no_defs +type=crs"
)
```
```

```
equal_earth_crs
mollweide_crs
````
```

### Next steps with these CRS objects

- **Trim poles**: restrict latitudes to  $\pm 89.5^\circ$  before warping.
- **Build target grid**: create a regular grid in the target CRS with 50 km cells.
- **Warp**: resample anomalies and surface layers with nearest neighbor.
- **Crop Equal Earth corners**: remove artifacts outside the round map.
- **Reusable function**: wrap these steps so you can call once with `equal_earth_crs` and once with `mollweide_crs`.

With `equal_earth_crs` and `mollweide_crs` defined, you're ready to plug them into a reusable function that trims, warps, and crops your raster for each projection.

Would you like me to sketch out the **function skeleton** next, showing how to take one CRS as input and return the warped/cropped stars object?